LESSON 06

Harness 设计 Superpowers 风格

Plan 是 AI 的外脑

把任务进度从上下文搬到文件里，让 /clear 也能续航

25 min

AI 驱动的软件开发提效实战课

`writing-plans` `executing-plans` `HANDOFF.md`

AGENDA · 议程

本节四个主题

01 **Harness 是什么**

AI 为什么会“变笨”

02 **四要素**

init / execute / checkpoint / handoff

03 **14 个 skill**

Superpowers 全景速览

04 **演示**

multi-period-backtest · 跨 /clear 续航

SECTION

01

Harness 是什么

AI 为什么会“变笨”

vibe 会话 vs harness 会话

维度	vibe 会话	harness 会话
进度记在哪	AI 的上下文里	plan 文件里
/clear 后会发生什么	一切重来	读 plan 接着干
任务上限	一个上下文窗口（≈ 200k tokens）	无上限
跨会话 / 跨人协作	不行	把 plan 路径给下一个人就行

harness 把任务从 AI 的“记忆”里拿出来，放进可审计、可移交、可断点续跑的文件。

一句话核心认知

把 state 从上下文里搬到文件里。

Move state out of the model's memory, into markdown.

上下文 = 短期记忆

AI 的所有 state 活在窗口里——满了或 /clear 就丢。

Plan 文件 = 长期记忆

磁盘上的 markdown，永远不丢；任何会话都能读它。

会话变工作台

可以随便清——只要进度都先写到 plan 里。

SECTION

02

Harness 的四要素

init / execute / checkpoint / handoff

init / execute / checkpoint / handoff

要素	干什么	文件落点
`init`	把任务拆成可独立验证的 step，写成 plan	**`plans/<task-name>.md`**
`execute`	按 plan 一步步跑，每跑出一笔就回填到 plan	修改 **`plans/<task-name>.md`**
`checkpoint`	跑一段后写交接快照（中间结果 + 下一步指针）	**`plans/HANDOFF.md`**
`handoff`	给下一个 session/ 人 /agent 一段整段可粘的 prompt	写在 **`HANDOFF.md`** 顶部

四个动作合起来 = 一份 harness，缺一个就漏一个环节。

三件套： plan / HANDOFF / report

plans/

<task>.md

过程记录

Goal · Architecture · Scenarios · Tasks · Resume Protocol

plans/

HANDOFF.md

会话交接快照

Status · 关键数字 · 一段整段可粘的 Next prompt

reports/

<task>.md

最终聚合结论

结果表 · Executive Summary · 3 句定性分析

Harness 不是工具，是约定。

三个目录保持同步，新会话第一件事永远是“读 plan 、找第一个 [ ]”。

SECTION

03

14 个 skill 速览

Superpowers 怎么把约定打包

Harness 三件套 — 本课的核心

`superpowers:writing-plans`

触发：有 spec / 需求、但还没动代码的任务起点

把多步任务展开成 plan 文件

Goal / Architecture / 进度表 / Tasks / Resume Protocol

`superpowers:executing-plans`

触发：拿到现成 plan，想让 AI 按计划落地

读一份 plan，按 step 跑、每跑完一步回填一笔

AI 在屏幕上一行一行 Edit plan

`superpowers:subagent-driven-development`

触发：plan 里任务彼此独立、可并行 / 不污染主上下文

每个 task 派一个新 subagent，跑完做 spec + quality 两段 review

保持主上下文整洁、可并行

三个加起来 = 本课讲的 harness；今天演示前两个。

思考与设计 + 实现纪律

3.2 思考与设计

sp:brainstorming

对话式问答把模糊想法磨成可写 spec 的设计

触发：你只有一个“想做 X”的念头

sp:writing-skills

用 TDD 的方式写 / 修 skill （skill 也是产物）

触发：想把高频流程沉淀成 SKILL.md

3.3 实现纪律

sp:test-driven-development

先写测试看挂、再写最少代码让过

sp:systematic-debugging

必须找到根因再动手，禁止症状修复

sp:verification-before-completion

声称完成前必须真跑验证，证据先于断言

评审与协作 + 环境与收尾

3.4 评审与协作

sp:requesting-code-review

派 code-reviewer subagent 看本次 diff

sp:receiving-code-review

收到 review 时先技术核实，不机械执行

sp:dispatching-parallel-agents

2+ 个独立任务派给不同 subagent 并行

3.5 环境与收尾

sp:using-git-worktrees

起独立 feature 前用 worktree 隔离工作区

sp:finishing-a-development-branch

实现 + 测试都过了，告诉你 merge / PR / 清理

sp:using-superpowers

插件元 skill — 告诉 AI 怎么找 / 用其它 skill

怎么触发 · 关键认知

方式 ① · 斜杠命令

`/superpowers:<skill-name>`

例：`/superpowers:writing-plans`

方式 ② · 自然语言

直接说要做的事，AI 自动匹配

“帮我把这个 5 步任务写成 plan” → writing-plans

关键认知

Skill 里没有代码逻辑，全是 prompt 工程。

你完全可以不装插件，自己手写一份“写 plan / 跑 plan / handoff” 的规范——效果一样。插件给的是规范的“门面”，真正起作用的是

纪律。

SECTION

04

演示： multi-period-backtest

writing-plans + executing-plans 跨 /clear 续航

三行口诀

state 不在脑子里，state 在 plan 里

跑一步，回一笔

HANDOFF 写得越具体，下一个会话越省心

贴在 CLAUDE.md 、写在 plan 模板顶部——一个月后再回看，价值远超 25 分钟。

三件事，记不下来其他都能忘

Plan 是 AI 的外脑

模型本身没有持久记忆——上下文是“短期记忆”，每次 /clear 就清空。

Plan 文件是“长期记忆”——磁盘上的 markdown，永远不丢。这一步认知翻转后，AI 才能干超出单上下文容量的长任务。

跑一步，回一笔

跑了 5 个 scenario 想着“等会儿一起填”——只要中间任何一次中断，那 5 个数字就活在 AI 脑子里没活在磁盘上。

纪律：跑一次就 Edit plan 一次，把数字 + 打勾一起改。这条规则没有例外。

Harness 不是工具，是约定

Superpowers 14 个 skill 里没有一个写了代码逻辑，全是 prompt 工程——把约定打包成 SKILL.md 。

你完全可以不装插件——只要手写一份格式相近的 plan 、并坚持每次会话开头说“读这份 plan 从第一个 [ ] 续”，效果一样。

三条要躲开的坑

1

Plan 写完不维护，跑到一半数字回填漏一行

跑完第 1 个填了，第 2 个想“等会儿一起填”，跑到第 3 个发现“等等第 2 个数字是多少来着”。

修复：每跑完一个 step 立刻 Edit plan，跑命令 / 看输出 / 回填当成一个原子动作。

2

Plan 太抽象，每个 step 都需要解释才能跑

[ ] Step 1: 把回测跑起来 — “跑起来”是空话；下一个 AI 看到也会回头问“具体怎么跑”。

修复：每个 step 必须含“做什么命令 + 看什么输出 + 改哪个文件”三段，让 plan 自包含。

3

HANDOFF 不指向具体下一步

“进度：Phase 1 跑完了，准备 Phase 2” — 下一会话看到要回头分析“Phase 2 是什么、怎么开始”。

修复：HANDOFF 里必须有一段 fenced 文本块，下一会话第一句整段粘就能续。

task1-harness · 30 分钟可完成

写一份 minimal plan，跑过一次 真的 /clear 续接。

3–5 step 的 mini plan 就够 — 练的是 harness 纪律，不是任务长度。

候选 ①

把仓库所有 README 翻译成英文（按文件分批）

候选 ②

把 lesson 4 backtest 跑更多策略（每 2 个一组）

候选 ③

把一个长函数重构成 4 个小函数（每次一个）

提交两件：

1. **`PLAN.md`**（最终版，全部 [x]）
2. 一段 200 字感受 · 续上的体验如何？plan 写到多详细才够？

完整说明： `lesson6/homework/task1-harness/README.md`

plan 跑得起来 → CI 守得住

把“每次都人工跑一遍 verify”升级成“在仓库里钉一道闸门”。

Lesson 6 教 plan 怎么写得让 AI 能续；lesson 7 教 verify 怎么写得让 CI 能挡——一前一后。

L1	L2	L3	L4	L5	L6	L7
Prompt 入门	上下文 + Skill	Plan & Spec	MCP	OpenSpec	Harness	Verify

L7 · 三节点验收 + 本地 /CI 双跑

`lint · pytest · review skill → bin/verify.sh + GitHub Actions`

AI 改的代码两道关——本地一分钟闭环，CI 兜底守 main。

本节到这里。

state 不在脑子里，state 在 plan 里。

跑一步，回一笔。

HANDOFF 写得越具体，下一个会话越省心。

下节 · Lesson 7 · 三节点验收 + CI